

What are External Tables?

An **External Table in Snowflake** allows users to **query data directly from external cloud storage without loading the data into Snowflake storage**.

The data remains in external storage such as:

- Amazon S3
- Azure Blob Storage
- Google Cloud Storage

Snowflake only stores **metadata about the files**, not the actual data.

This allows organizations to **analyze large datasets stored in data lakes without moving them into Snowflake**.

Key Characteristics of External Tables

- Data remains in **external cloud storage**
 - Snowflake stores only **metadata**
 - Supports **semi-structured formats**
 - Enables **data lake analytics**
 - No need to load data using COPY INTO
-

Supported File Formats

External tables commonly work with:

Format Type

Parquet Columnar

JSON Semi-structured

Avro Row based

ORC Columnar

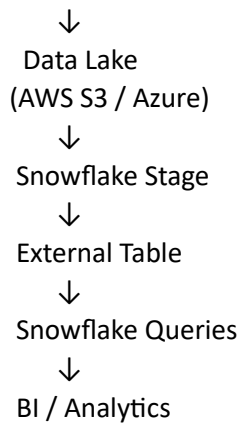
CSV Structured

However, **Parquet is the most efficient format for external tables**.

External Table Architecture

Typical architecture in modern data platforms:

Application / Data Sources



In this architecture:

Snowflake **does not store the data** but **queries it directly from the data lake**.

Real-Time Project Scenario

Industry: **E-commerce Analytics Platform**

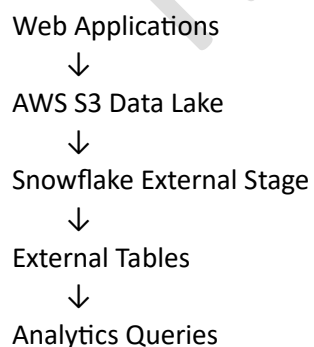
The company stores **large historical data in S3 Data Lake**.

Example datasets:

Dataset	Storage
Website logs	JSON files
Order transactions	Parquet files
Customer activity	Parquet files

Instead of loading terabytes of data into Snowflake, they use **External Tables to query data directly**.

Pipeline architecture:



Step 1 – Create Storage Integration

First create **secure connection to cloud storage**.

Example:

```
CREATE STORAGE INTEGRATION s3_int
TYPE = EXTERNAL_STAGE
STORAGE_PROVIDER = S3
ENABLED = TRUE
STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::123456:role/snowflake_role'
STORAGE_ALLOWED_LOCATIONS = ('s3://company-data/');
```

This allows Snowflake to **access S3 buckets securely**.

Step 2 – Create External Stage

The stage defines **location of files in cloud storage**.

Example:

```
CREATE STAGE ext_stage
URL='s3://company-data/orders/'
STORAGE_INTEGRATION = s3_int;
```

Now Snowflake can access files stored in **S3 bucket**.

Step 3 – Create File Format

Define the file format.

Example for Parquet:

```
CREATE FILE FORMAT parquet_format
TYPE = PARQUET;
```

Example for JSON:

```
CREATE FILE FORMAT json_format
TYPE = JSON;
```

Step 4 – Create External Table

Now create an **External Table** referencing the files.

Example:

```
CREATE EXTERNAL TABLE ext_orders
(
  order_id INT AS (value:order_id::INT),
  customer_id INT AS (value:customer_id::INT),
  amount NUMBER AS (value:amount::NUMBER)
)
WITH LOCATION = @ext_stage
FILE_FORMAT = parquet_format;
```

Snowflake now **maps columns to file data**.

Querying External Tables

Example query:

```
SELECT *
FROM ext_orders
WHERE amount > 500;
```

Snowflake reads the data **directly from S3**.

Metadata Storage

External tables store **file metadata** such as:

- File location
- File format
- Column mappings

Actual data remains in **external storage**.

Refreshing External Tables

When new files arrive in the data lake, Snowflake must **refresh metadata**.

Example:

```
ALTER EXTERNAL TABLE ext_orders REFRESH;
```

This updates the metadata to include **new files**.

Partitioning External Tables

External tables support **partitioning to improve query performance**.

Example partition structure in S3:

```
s3://company-data/orders/  
year=2024/  
month=01/  
day=01/
```

Create external table with partitions:

```
CREATE EXTERNAL TABLE ext_orders  
(  
  order_id INT,  
  customer_id INT,  
  amount NUMBER  
)  
PARTITION BY (year, month)  
LOCATION = @ext_stage  
FILE_FORMAT = parquet_format;
```

Partition pruning improves query performance.

External Tables vs Internal Tables

Feature	External Table	Internal Table
Data Location	Cloud storage	Snowflake storage
Storage Cost	Low	Higher
Performance	Slightly slower	Faster
Use Case	Data lake analytics	Warehouse analytics

Advantages of External Tables

- No data duplication
- Lower storage cost
- Access large datasets in data lakes
- Works well with big data pipelines

Limitations

- Query performance slower than internal tables
- Requires metadata refresh
- Not ideal for high-performance BI queries

Best Practices

In production environments:

- Store data in **Parquet format**
- Use **partitioned file structures**
- Refresh metadata regularly
- Use external tables for **data lake exploration**

Real Production Example

Company stores **5 years of transaction history** in S3.

Instead of loading all data into Snowflake:

- Recent data → Snowflake tables
- Historical data → External tables

Example query combining both:

```
SELECT *  
FROM recent_sales  
UNION ALL  
SELECT *  
FROM ext_sales_history;
```

This approach balances **performance and cost**.

Common Interview Questions

What is an External Table?

An external table allows Snowflake to **query data stored in external storage without loading it into Snowflake**.

Where is the data stored?

In **cloud storage like S3, Azure Blob, or GCS**.

What does Snowflake store for external tables?

Only **metadata about the files**.

Which file format is best for external tables?

Parquet

How do we detect new files in external tables?

Using:

```
ALTER EXTERNAL TABLE REFRESH;
```

Quick Summary

External Tables enable Snowflake to **query data directly from external cloud storage**, making it possible to analyze large data lake datasets without physically loading them into Snowflake.

They are commonly used in **data lake architectures and big data environments**.